

需求管理系统的分析、建模与实现

——从受控需求到多智能体辅助改进

姓名：黄展鹏 学号：2244413942

专业班级：软件2404班

2026 年 10 月 4 日

摘要

这次作业要求分析需求管理系统中的实体、属性和相互关系，并利用多智能体辅助完成实现、检查和重新设计。我的分析围绕几个核心问题展开：需求管理与项目管理怎样区分，一条需求需要保存什么，评审、版本和变更为什么必须独立，以及 AI 生成的代码怎样经过检查后才可以形成改进结论。围绕这些问题，我比较了直接覆盖需求与保留正式快照、统一状态与独立变更流程、微服务与模块化单体等方案。

设计的重点是保留“当时确认了什么”：当前需求、正式版本和每轮评审快照各有用途，批准变更也与应用变更分开。完整逻辑 ER 图将所有实体、字段和外键放在同一张 A3 横向页中；实现使用 Vue 3、Spring Boot 和 MySQL。随后，我把需求追踪、架构、实现、测试、审查和重构分成不同逻辑角色，用相同的 32 个代表性场景比较两版代码。第一版为 31 项通过、1 项失败；依据四个真实问题形成四条约束后，第二版为 32 项通过。这个结果仅支持已检查范围内的改进，不能代替未运行场景或生产环境验证。

系统源码、完整 ER 模型、补充测试与实现说明：

<https://github.com/hechuxielianli/xjtu-rms-coursework>

目录

1 题目分析	3
1.1 我怎样理解题目 2.1	3
1.2 需求管理和项目管理的边界	3
1.3 我怎样理解题目 2.2	3
2 需求管理系统的需求分析	4
2.1 一条需求需要保存什么	4
2.2 使用者和权限	5
2.3 需求从提出到确认的过程	5
2.4 正式需求为什么需要评审、版本和变更	5
2.5 主要用例与业务规则	6
3 ER 模型设计	7
3.1 我怎样从业务事实抽取实体	7
3.2 完整 ER 图	7
3.3 几组关键关系为什么这样设计	9
4 系统设计	10
4.1 两套状态机	10
4.2 系统架构选择	11
4.3 版本、权限和事务如何落到实现	12
4.4 系统界面与主要功能	13
5 多智能体辅助实现	15
5.1 单一 AI 工作方式的问题	15
5.2 我的多 Agent 分工	16
5.3 第一版是怎样产生的	16
5.4 我怎样避免 AI 自己验证自己	17
6 第一版代码的实际检查	17
6.1 测试选择	17
6.2 第一版结果	17
6.3 四个有代表性的问题	18
6.4 这些问题为什么会出现	19
7 受约束的第二次设计	19
7.1 从四个问题得到四条约束	19
7.2 第二版做了什么	20
7.3 V1 / V2 对比	20
8 我的结论	21

1 题目分析

1.1 我怎样理解题目 2.1

题目 2.1 先要求定义需求的属性特征，再对整个需求管理系统的实体、属性和关系建模，最后设计系统。这三个要求并不是三个互不相关的文档。属性决定一条需求有哪些可检查的信息，实体和关系决定这些信息怎样保存，系统设计则需要回答谁在什么条件下可以修改它们。如果只画表格而不解释业务过程，图中即使有很多关系线，也不能说明模型合理。

因此，我先把题目中的“管理”理解为：需求怎样提出、怎样被理解和确认、变化以后怎样保持可追踪。一个只支持增删改查的文本库能够保存描述，却无法说明评审者批准了哪份内容，也无法说明当前需求为什么与原先不同。版本、评审和变更由此成为需要推导的对象，而不是为了增加实体数量而添加的表。

题目允许参考已有系统。我参考教材的数据建模方法，并查看 IBM DOORS Next 对属性和追踪关系的说明[1, 2, 3]。实际产品提示了需求可以有结构化属性和有类型的链接；但完整产品还面向更广泛的工程协作。后文中的固定分类、五个角色和单项目范围，是本课程系统的具体选择，不能当作对该产品的完整复现。

1.2 需求管理和项目管理的边界

这里首先要解决的是管理对象的范围。如果加入 **Project**，就需要继续解释项目成员、里程碑、迭代和进度；再加入 **Sprint** 与 **Task**，需求就容易变成任务的附属说明。这样的系统可能有用，但其主要问题会变为“谁在什么时候完成工作”，而本题首先问的是“需要什么，以及这个需要怎样被确认和修改”。

两者当然有交集。需求可以影响计划，也可以由实现任务落实；但本次建模把一个项目当作隐含的运行边界，不把排期、代码仓库和完整测试管理放入领域模型。这样做的代价是不能管理多项目隔离和项目进度，收益是可以完整讨论需求自己的生命周期。没有 **Project** 实体，是范围选择的结果，不能据此声称系统已经覆盖项目管理。

1.3 我怎样理解题目 2.2

题目 2.2 不是只要求“让 AI 写出代码”。它同时要求协作架构、代码质量和缺陷分析、原因解释，以及受控限制下的重新设计。若几位 Agent 仅分别完成前后端，然后由同一个生成过程宣布成功，仍然缺少独立的判断依据。

我据此把实现当作检验前面设计的手段：先确定业务预期，再产生第一版；检查时固定实现，避免一边运行一边悄悄修补；问题接受以后，把根因写成能够落实和检查的约束，再在相同场景下观察第二版。本文中的应用 V1、V2 指两次代码实现；需求自身的正式版本也有版本号，两者在后文分开讨论。

把题目拆开以后，我觉得后面的设计围绕几件事展开。首先要弄清需求管理到底管理什么，否则很容易把系统做成项目管理；接着要回答一条需求在被评审、批准和修改时，哪些信息必须保留下来。这些信息的归属和用途，直接决定了实体与关系。再往后，则是怎样把这些规则落实到状态转换和程序结构中，让模型中的约束成为实际操作的边界。

题目第二部分又多了一层要求：AI 不仅要帮助实现，还要能够暴露第一版的问题，并让后续修改受到明确约束。因此，后面的分析分别从业务模型、系统实现和两版代码比较展开；实现和检查的结果用于检验前面的设计，也为进一步修改提供依据。

2 需求管理系统的需求分析

2.1 一条需求需要保存什么

一条需求如果只有标题和描述，记录起来最方便，但评审者很难判断它来自谁、为什么需要，以及实现到什么程度才算满足。教材把需求管理与变更、追踪联系起来[1]；这提示我需要保存可供后续决策的信息。表1给出了本系统的八项正式内容。这些属性不是越多越好，而是分别回答一个实际问题。

表 1: 八项正式内容及其用途

属性	为什么需要
标题、描述	快速识别需求，并说明需要具备的行为或条件。
层级	区分业务、用户和系统三个观察层次。
性质	区分功能、质量要求和约束，避免不同标准混在一起。
优先级	在需求之间表达处理次序；本系统不据此生成项目排期。
来源	说明需求从什么业务事实、人员或材料提出。
理由	解释提出它的原因，给后续变更判断提供依据。
验收标准	把模糊描述转成可以观察的满足条件。

分类需要特别区分两个维度。“业务需求、用户需求、系统需求”说明抽象层次；“功能、质量、约束”说明要求的性质。若把它们混成一组，系统需求和功能需求就会被误当作互斥选项。接口需求也可能是功能要求或约束，因此“接口”用于标签分类，不再占据性质枚举。这里的层级值为 BUSINESS/USER/SYSTEM，性质值为 FUNCTIONAL/QUALITY/CONSTRAINT。

草稿允许逐步补齐来源、理由和验收标准。若创建时要求所有内容完全成熟，用户可能在系统外保存不完整需求，反而失去讨论过程；但如果提交时仍不检查完整性，评审就没有统一对象。因此完整性检查放在“提交评审”这个动作上，草稿和可评审内容的边界由业务动作表达。

2.2 使用者和权限

另一个选择是把权限直接绑定每个账号，或用预定义角色表达职责。逐账号授权灵活，却需要解释大量权限组合；课程系统的职责比较稳定，所以使用表2中的五种角色。一个用户可以有多个角色，权限取并集，但自我评审限制始终有效。

表 2: 角色及主要职责

角色	可以执行的主要操作
管理员	管理用户、启用状态和角色分配，查看需求、版本及审计。
需求工程师	维护草稿和管理信息、建立关系、提出并应用批准的变更。
评审者	对有效需求或变更评审轮次作决策，参与评论。
项目成员	对当前正式版本确认实现和验证，参与评论。
查看者	查询需求、关系、版本和可见历史，不执行修改。

如果把管理员理解为可以绕开所有业务规则的人，创建者给自己增加评审角色就能批准自己的需求，评审分工会失效。因此角色并集解决“可以做哪些操作”，创建者或提出者身份限制解决“这次操作能否由此人执行”。前端可以隐藏不适合的按钮，但后端仍要用可信会话中的用户身份检查；客户端传入的操作者或角色不能成为授权依据。

用户禁用与删除也不同。删除账号会让旧评审和版本失去作者，禁用则停止后续登录和操作，同时保留已有引用。系统保存密码哈希，不把明文密码、哈希或会话凭据返回到业务页面。管理员修改基本资料也不会顺便改变密码。

2.3 需求从提出到确认的过程

以“用户账号登录”为例，草稿最初可以说明用户需要进入系统，但还没有完整的验收标准。需求工程师补充来源、理由及成功和失败条件以后提交。系统此时创建一个评审轮次，评审者面对的是提交瞬间的内容，而不是之后可能改变的页面文本。

评审有三个结果：批准、拒绝、要求修改。要求修改意味着问题可以继续补充，需求回到草稿；拒绝意味着这次提交没有被接受，需求进入已拒绝，工程师可以重新打开后再讨论。两者都保存原意见。首次批准则形成正式 V1，随后项目成员分别确认当前版本已实现、已验证。批准只是认可要求，不能替代实现或验证。

这个过程也解释了为什么不让用户任意编辑 `status`。一个普通字段修改无法同时表达“检查完整性、建立快照、完成轮次、创建版本”等后果。把提交、批准和应用变更设计成独立动作，增加了一些接口，但让状态变化能够对应具体业务事实。

2.4 正式需求为什么需要评审、版本和变更

这里有两个容易混淆的选择。其一，批准后继续直接更新需求行；其二，批准内容形成不可变版本，后续变化经过申请和批准。前者只要一个编辑入口，代价却是旧

的批准内容消失，无法回答“评审者当时确认了什么”。因此当前需求保存工作状态，正式版本保存八项内容的完整快照、来源、产生者和原因。

只保存评审者、意见和决策仍不够。需求后来被修改时，旧意见如果读取最新内容，会看起来像是评价了从未见过的内容。每次提交因此还要保存完整评审快照。评审快照记录一次提交，即使被退回也存在；正式版本记录被认可的内容，仅在首次批准或批准变更应用后形成。这两个对象不能合并。

批准后若需要增加邮箱登录，申请者应说明修改理由和建议内容，并明确依据哪个版本。直接改当前需求最简单，却使正式版本失去控制作用。因此变化由 `ChangeRequest` 承载，其评审由 `ChangeRequestReview` 承载。申请批准表示允许这一变化；显式应用才把它变成新正式版本，避免审批动作无声地修改现行要求。

负责人、标签和当前关系则用于组织与查找，不属于八项正式内容。例如把登录需求的负责人从甲改为乙，不代表登录行为本身改变。这些管理信息修改留下审计，不生成正式版本。这样的区分降低了版本噪声，代价是正式内容历史和管理历史要从不同入口阅读，界面需要把它们说明清楚。

2.5 主要用例与业务规则

业务规则需要说明例外和前提，不能仅列功能名称。表3概括十二个核心用例。完整功能需求和规则清单在公开仓库中；本文给出理解设计所需的关键约束，读者不打开仓库也能理解后面的模型和实验。

表 3: 十二个核心用例的分析重点		
编号	用例	主要前提或后果
01	用户认证	账号启用，凭证匹配，登录成功才建立会话。
02	管理用户与角色	管理员执行，预定义角色有效，旧历史引用保留。
03	创建与维护需求	仅草稿编辑内容，管理信息另行维护。
04	推进需求生命周期	通过具体动作转换，提交建立新轮次。
05	评审需求	禁止创建者自评，首次批准原子形成正式 V1。
06	查询与追踪需求	筛选、分页、详情和有界追踪，不改变需求。
07	管理需求关系	不得自关联、对称重复或形成依赖环。
08	查看版本与历史	版本只读，首次批准前说明无正式版本。
09	协作与评论	作者可以逻辑删除本人评论，保留删除历史。
10	管理变更请求	有当前基线且无活动变更，每次提交保存新快照。
11	评审变更请求	禁止提出者自评，批准不直接生成需求版本。
12	应用批准变更	基准仍为当前版本，同一事务生成新版本并结束变更。

分析整理为 64 条功能需求和 97 条业务规则。它们的价值在于明确上述前提，并为实现和检查提供共同预期，不在于数量本身。比如“查看历史”不能被映射成修改历史的入口，“管理员查看版本”也不能解释为管理员可以改旧版本。非功能目标包括服务端权限检查、不可变历史、事务完整性、冲突反馈、可追踪关系和稳定分

页；本次以相应反例、事务故障场景和查询计数检查其部分内容，不把这些设计目标直接宣称为生产指标。

3 ER 模型设计

3.1 我怎样从业务事实抽取实体

题目中心对象是需求，所以先有 Requirement。但它的一行记录只能表达当前内容和生命周期。正式内容需要保留，因此推导出 RequirementVersion；提交内容需要与评审结果绑定，因此又推导出 RequirementReview。这三者分别回答“现在是什么”、“已确认过什么”和“这一轮提交过什么”。

正式内容不允许直接改写后，修改理由、基准和建议内容就需要独立载体，得到 ChangeRequest。变更本身可以被退回并重新提交，不能只在申请上覆盖最后一条意见，于是得到 ChangeRequestReview。两类评审字段虽然相似，业务对象和批准后果却不同；若合为一张带任意对象类型的评审表，外键和内容完整性会更难表达。本次选择两个明确实体，承担少量结构重复，换取清楚的关系和检查条件。

需求之间还有依赖、冲突、细化、派生、重复和一般关联。一条需求可以关联多条需求，同一对需求也可能具有不同意义，因此用 RequirementRelation 表示带类型的递归多对多关系。源端与目标端各引用需求，而不是在需求上放一个逗号分隔的编号列。

标签可被多条需求共享，一条需求也可以有多个标签。标签目录用 Tag 表示，多对多关联由 RequirementTag 表示。评论有自己的作者、内容和删除历史，所以还需要 Comment。

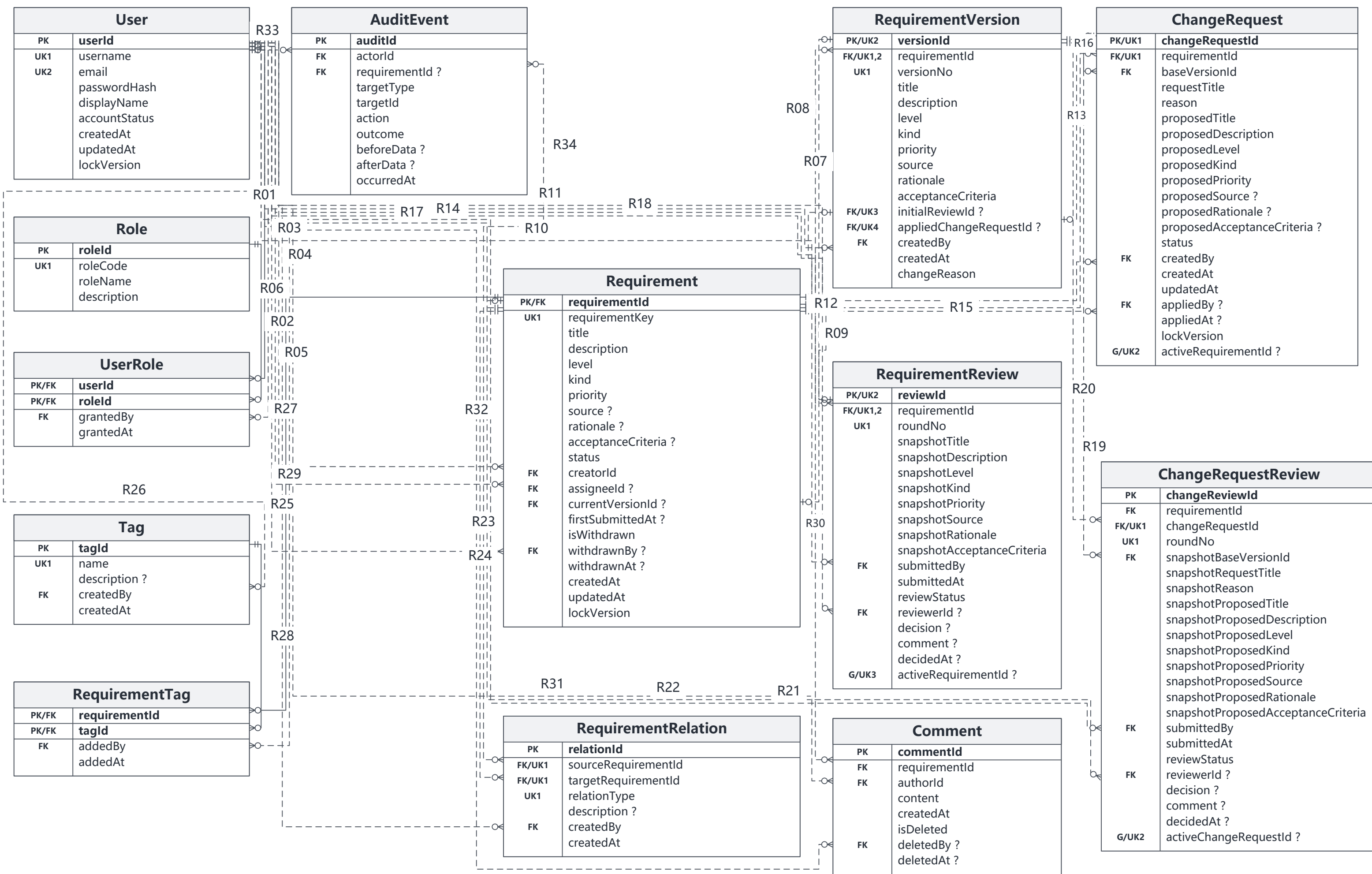
实际操作者需要账号，而职责可以组合，因而用户、角色及其关联分别由 User、Role 和 UserRole 表示。关键操作还需要说明谁改变了什么，得到 AuditEvent。这样从九步业务推导得到十三个实体；实体数是分析结果，不是论证的起点。

3.2 完整 ER 图

图1将十三个实体的 151 个字段和 34 个真实外键关系放在同一张 A3 横向画布。实体以表框表示，字段逐行列出，PK、FK 和唯一键组清楚标记；问号表示可空，G 表示由数据库生成的约束列。相同 UK 编号在同一实体内构成一个唯一键组，不能把复合唯一键的各列误读为分别唯一。

图中的关系线使用 Crow's Foot 符号，实线表示标识关系，虚线表示非标识关系。父端为 1 或 0..1，子端为 0..N 或 0..1；R01-R34 只用于在补充外键表中定位关系。当前版本、首次评审和已应用变更之间的部分引用是复合外键，它们同时约束记录归属。通用审计目标的类型和编号只做逻辑定位，不被画成通向所有实体的虚假数据库外键。

图1 需求管理系统完整逻辑 ER 图



3.3 几组关键关系为什么这样设计

3.3.1 稳定需求标识与正式版本

需求编号要持续代表同一条需求，版本号则说明这一需求的哪次正式内容。把需求编号随版本变化，会让评论、关系和评审很难聚合；只用版本号，又无法区分不同需求的 V1。所以需求和版本分开，版本属于一个确定需求，版本号在该需求内唯一且连续。

首次批准前，需求可以没有正式版本；形成基线后，当前版本指针必须引用属于自己的版本。因此既需要可空当前指针，也需要由需求标识和版本标识组成的归属约束。不能因为某个版本标识真实存在，就允许把甲需求的当前版本指向乙需求。

3.3.2 评审轮次与内容快照

需求可以有零至多个评审轮次，每轮必须属于一条需求。草稿还没有评审时允许为零，提交以后又要防止两个进行中轮次同时作决定。普通外键解决归属，活动轮次唯一约束和服务端状态检查共同解决“一次提交只能有一个有效待决策对象”。

快照增加了内容副本，但这里的副本有明确用途：它保留时间点上的事实。后续编辑不得回写旧快照，完成的意见和决策也不得覆盖。如果只存差异，历史还原需要依赖全部前序内容；课程系统选择完整快照，用一定存储量换取独立阅读和可靠比较。

3.3.3 基准、变更和新版本

一条需求可以保留多个历史变更，但同时只能有一个活动变更。申请引用当前基准版本，每轮变更评审再保存这个基准、申请标题、理由和八项建议内容。批准以后仍要检查基准是否过期，才允许显式应用。这组关系把“允许改变”和“当前内容已经改变”分开。

新版本既保留来源变更，又属于原需求；应用完成后变更记录标为已应用。两边的关联使追踪可以从版本回到修改理由，也可以从申请找到实际结果。由于这些记录同事务产生，插入顺序和最终更新需要由应用服务协调，不能指望外键自动完成整个业务动作。

3.3.4 关系、标签与用户职责

`RequirementRelation` 的两端分别引用需求。同一条需求可以有多条入向和出向关系，源和目标不是两个新实体。对称关系只保存规范化的一份，防止甲关联乙和乙关联甲被重复计数；依赖关系具有方向，还要检查添加后是否产生环。

标签分成目录与关联表，避免在每条需求内复制标签名称。用户和角色也用关联表，避免把多个角色写进一个字符串。需求创建者必须存在，负责人则可以暂时未分配；禁用用户不删除这些关联。这样的可空性来自业务事实，不能为简化表格把所有引用统一设为可空或非空。

3.3.5 评论、审计与受控冗余

评论是讨论内容，审计是操作记录，两者不能互相替代。删除评论用逻辑标记保留作者和时间；需求撤销也保留历史，只对从未提交且已解除有效关系的草稿开放。审计保存操作者、动作、结果和必要的前后变化，但不把密码、哈希或会话凭据写入变化文本。

用户角色、标签关联和独立评审减少了多值属性及更新异常。不过不能笼统说模型完全没有冗余：需求当前内容、正式版本、评审快照有意重复八项内容。区别在于副本分别属于当前、已批准和已提交三个语义，复制只能通过规定动作完成。规范化组织当前关系，受控冗余保留历史，两者在这个场景中共同起作用。

4 系统设计

4.1 两套状态机

如果把需求和变更都压进一个状态字段，需求“已批准”可能既指内容形成基线，又指某次修改获得批准。后续应用就难以判断到底批准了什么。因此它们各有状态机。需求有草稿、待评审、已批准、已实现、已验证和已拒绝；变更有草稿、待评审、已批准、已应用、已拒绝和已取消。撤销是需求的独立逻辑标记，不增加一个生命周期状态。

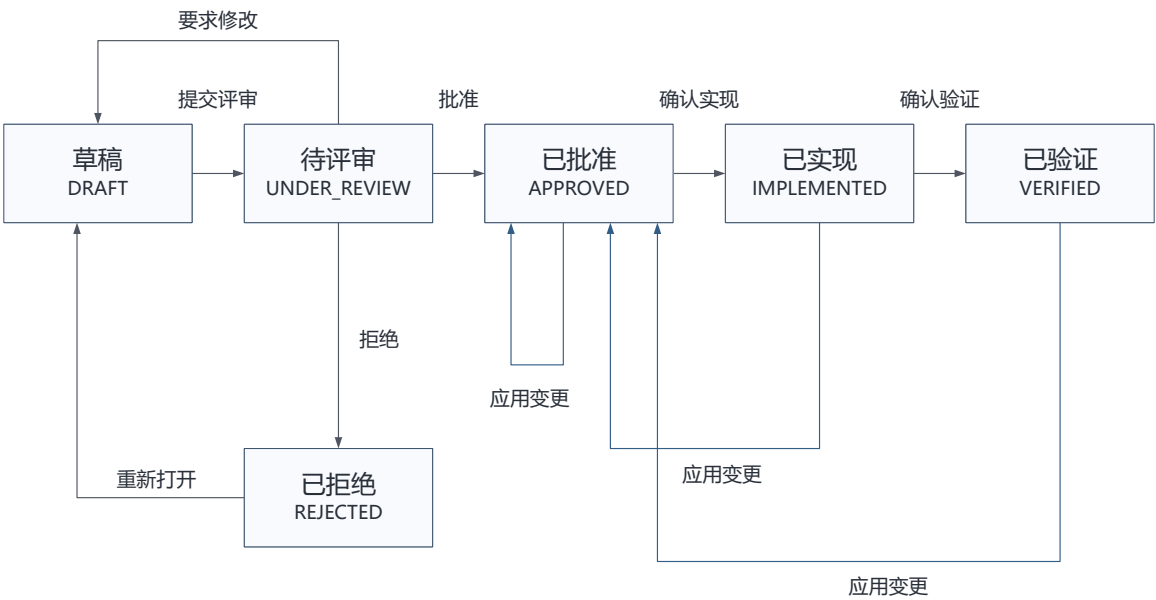


图 2: 需求状态：新正式内容需要重新确认实现和验证

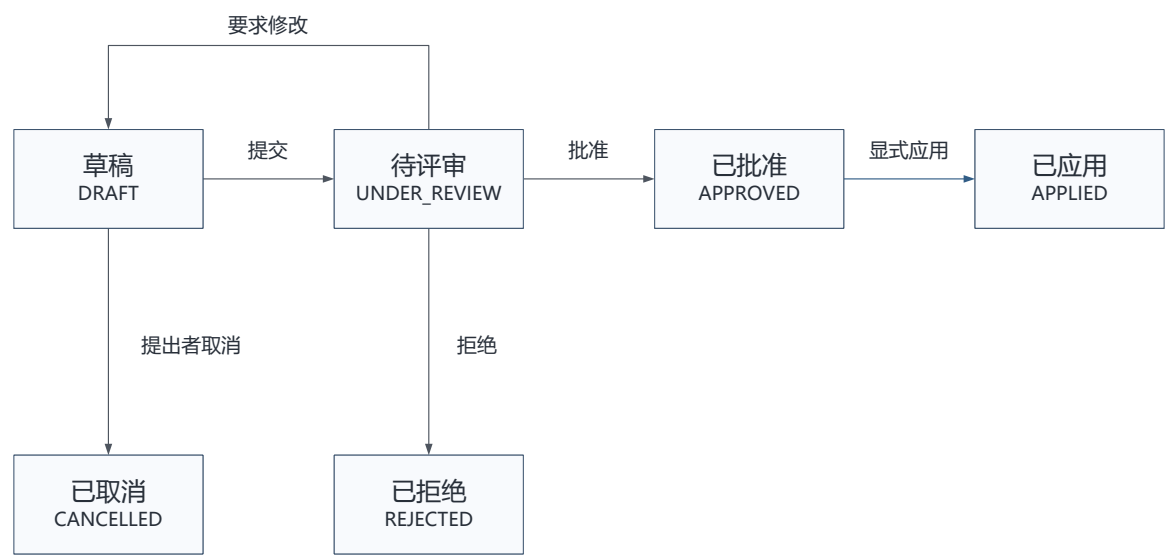


图 3: 变更状态：批准与应用分开

图2中，首次批准创建需求正式 V1。批准、实现或验证以后应用变更，需求统一到已批准，不能把旧版本的实现、验证结论自动移给新内容。图3中，变更被要求修改后回到草稿，再提交形成新轮次；拒绝、取消和已应用是终态，不能通过直接写字段重新提交。批准变更仍不创建版本。

状态机只是合法转换的图形表达。实际动作还要检查权限、未撤销、内容完整性、自评限制、评审轮次和修订号。图上有箭头，不能解释为任意用户都能沿它操作。

4.2 系统架构选择

微服务可以把业务边界部署成多个服务，但本系统的一次变更应用会同时修改版本、需求、变更和审计。如果它们各有数据库，就要额外处理跨服务失败和一致性。这个复杂度并不直接回答课程题目，也容易掩盖实体关系和事务边界。

因此我选择模块化单体：Vue 3 承担页面和交互，Spring Boot 在一个进程内按用户、需求、评审、版本、变更、关系和审计分工，MySQL 保存关系数据。各模块有明确接口，但关键写动作仍可以使用同一个数据库事务。它的代价是部署和扩容不能完全独立，收益是课程范围内的业务流程更容易检查，故障也更容易完整回滚。

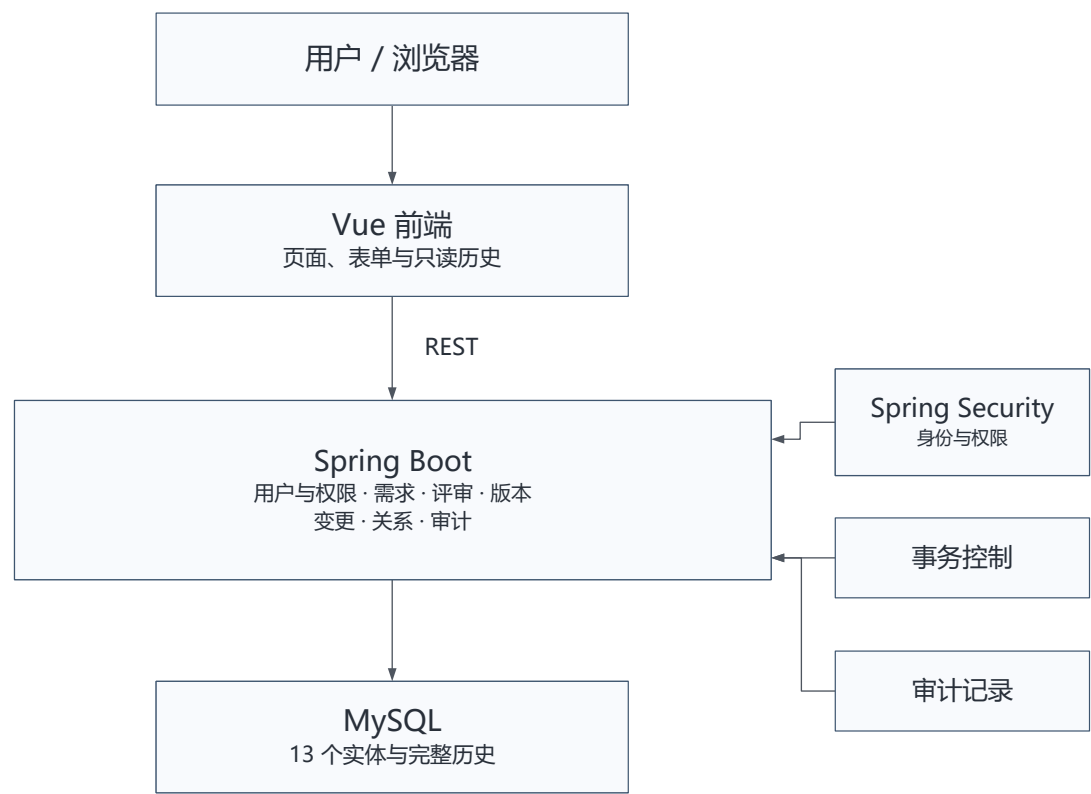


图 4: 课程系统的前端、业务服务和数据库

页面接收输入，Controller 将请求转成动作参数，应用服务协调状态、权限和事务，领域规则决定允许的变化，持久化层读写数据。数据库表不直接作为前端响应，避免页面顺着实体关联取得不应公开的字段。Spring Security 处理会话和授权入口，业务服务继续检查创建者、提出者和对象状态等具体限制。

4.3 版本、权限和事务如何落到实现

4.3.1 把字段编辑和业务动作分开

内容编辑、负责人和标签修改、提交、决策、应用使用不同接口。这样客户端提交的字段不会顺便改变状态和版本。表4列出主要动作的接口含义；完整 OpenAPI 在公开仓库中，本文只保留理解设计所需的动作边界。

表 4: 主要接口及其业务边界

方法	路径示意	含义
POST	/requirements	创建草稿，标识由服务端生成。
PATCH	/requirements/{id}/content	仅编辑草稿八项内容。
PATCH	/requirements/{id}/metadata	修改管理信息，不形成版本。
POST	/requirements/{id}/submit	检查并建立评审轮次、快照。
POST	/requirement-reviews/{id}/decision	决策有效轮次，批准首次形成 V1。
POST	/changes/{id}/submit	提交变更并复制完整申请快照。
POST	/change-reviews/{id}/decision	只批准申请，不应用内容。
POST	/changes/{id}/apply	检查基准并原子产生新版本。

路径示意省略统一 API 前缀。操作者来自会话；角色、状态、版本号和审计时间由服务端确定。修订号用于判断用户是否仍基于自己刚读到的对象操作。旧修订不能覆盖已经改变的数据，冲突返回明确结果，页面重读以后再决定如何处理。

4.3.2 一次应用的事务边界

考虑一个反例：先插入新版本，随后更新需求失败。如果两步各自提交，版本列表里有 V2，当前指针却仍是 V1，用户会看到相互矛盾的结果。因此应用变更的完整动作包括：锁定需求与变更；复核批准状态和基准；追加下一版本；复制批准内容并更新当前指针；把需求置为已批准；把变更置为已应用；写入成功审计。这些步骤同事务提交，任一步失败都不能留下部分成功。

数据库负责真实外键、唯一键和活动占用约束，业务服务负责权限、自评、合法转换、连续版本号和依赖环等语义。并发创建两个活动变更时，不能只靠“先查询再插入”；串行化与数据库约束共同避免两个请求都认为没有活动变更。重复审批和应用也需要在锁定后再次检查，不能依赖按钮只被点一次。

成功审计与业务修改同事务。失败审计则在主事务结束后独立记录，否则主事务回滚会把失败记录一同删除。这是技术记录与业务数据的不同需求，不能为了减少代码就把两者随意合并。拒绝或失败记录还必须保留原异常含义，不能把已经提交的操作错误地记成未提交。

4.4 系统界面与主要功能

界面应帮助使用者区分当前内容、正式版本和历史提交。需求工作区提供筛选、分页和详情，详情用概览、版本、评审、关系、变更、评论和历史页签组织内容。状态和当前版本放在管理信息区，描述、理由与验收标准放在正式内容区。

图5与图6来自独立展示数据库：前者说明需求内容怎样阅读，后者说明基准、建议内容和应用结果怎样对应。展示用户与自然需求用于说明页面，不是第一版测试现

场，也不是实验通过证据；“邮件通知”“需求导出”等示例标题不表示系统新增了对应产品功能。当前前端只调整布局、CSS、显示组件和文案，业务接口、路由、权限与动作保持原有语义。

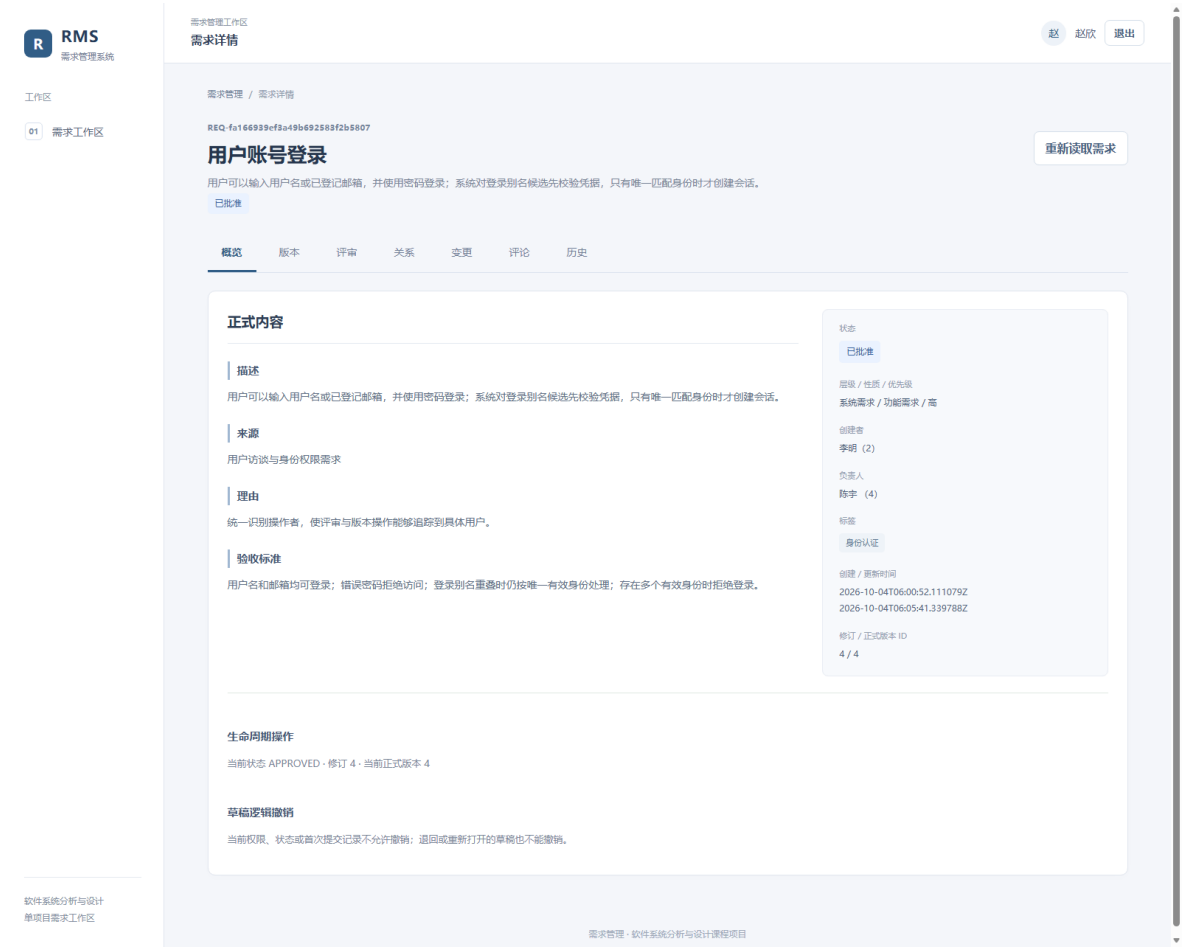


图 5: 需求详情：正式内容与管理信息分开呈现



图 6: 变更申请: 从基准版本到建议内容，再到批准和应用

5 多智能体辅助实现

5.1 单一 AI 工作方式的问题

如果同一个 AI 先解释需求、再写代码、最后用自己理解的代码写预期，它可能把一次错误理解同时带到实现和测试中。测试全部通过只能说明两者一致，未必说明它们符合题目和业务规则。人工也会有这一问题，因此这里并不是把错误简单归因于某个模型。

拆分角色的目的，是让预期从需求和设计中产生，让检查能够指出实现偏离它的地方。角色分离会带来交接成本，也不能保证不同角色没有共享假设，所以仍需要我的范围判断、原始结果留存和具体证据。多智能体的价值应从这些工作说明，不能用 Agent 数量作为可靠性指标。

5.2 我的多 Agent 分工

实际协作包含需求追踪、架构、前后端实现、测试、审查和受约束重构几个逻辑角色。表5强调各自依据什么作判断；图7只表示实际工作顺序，不把内部编号和过程文件塞进课程解释。

表 5: 协作角色及相互检查的关系

角色	职责与判断依据
需求追踪	把需求、规则、用例和检查场景对应起来，发现遗漏和歧义。
架构	说明模块接口、状态守卫、事务和数据边界。
后端、前端实现	按共同设计实现，不能自改需求预期。
测试	用既定场景运行真实 HTTP、浏览器和数据库，保留失败。
代码审查	只读分析代码路径、根因及范围，区分观察与推测。
重构	依据被接受问题和明确约束修改，再接受同一场景检查。



图 7: 从共同设计到独立检查，再到受约束的改进

5.3 第一版是怎样产生的

实现开始前，需要共同解释评审快照、正式版本和变更应用。如果后端把批准变更理解为马上生成版本，而前端把它理解为等待显式应用，两边都可能各自编译成功，却无法连成同一业务过程。因此需求和架构角色先确定输入输出、动作前提和后果，前后端再按这些约定分工。

实现按用户权限、需求维护、评审版本、变更和追踪协作逐步形成可运行流程。开发检查帮助及时发现类型、接口和模块依赖问题；它们属于生成阶段的反馈，不能

与后续独立实验混为同一批计数。第一版固定下来以后，才用代表性场景观察它的实际行为。

5.4 我怎样避免 AI 自己验证自己

我把业务预期与第一次实现分开保存，测试阶段不让实现角色边运行边修改。测试报告记录实际请求、页面旅程和数据库后果，代码审查再解释这些观察与代码分支的关系。一个静态风险不能自动算作运行失败，一个测试失败也需要排除环境或脚本定位错误。

问题接受以后，重构角色只处理明确根因，第二版仍使用原场景和预期。这个安排不能证明所有缺陷都被发现，但可以避免通过删失败测试、放宽预期或更换计数口径制造“改进”。我在这里负责的是判断证据够不够，以及修改有没有越过已经确定的业务边界。

6 第一版代码的实际检查

6.1 测试选择

完整设计列出 103 项检查，但本次实际执行 32 个代表性原设计场景。我选择它们时，优先覆盖身份和权限、需求提交和决策、正式版本、变更应用，以及事务、并发和两个完整浏览器旅程。另有协议、认证边界和查询计数补充检查，单独统计。

这不等于另外 71 项可以忽略。它们仍然是未运行，部分代表场景内的未覆盖主体也继续披露。选择代表场景能够先检查核心闭环，代价是结论范围有限；若把开发阶段所有单元断言都加进来，得到更大的“通过数”，反而失去两版同口径比较的意义。

检查不只看 HTTP 是否成功。例如应用变更以后，需要同时确认当前指针、正式版本八项内容、申请状态和审计；自我评审即使按钮不存在，也要通过直接请求验证后端拒绝。事务故障场景则观察失败以后是否完整回滚，避免只根据异常响应推断数据库状态。

6.2 第一版结果

应用 V1 的 32 个代表场景中，31 项通过，1 项失败，阻塞为零。两条浏览器旅程和已执行的状态、权限及事务反例未显示业务结构被破坏，但这不能替代未运行范围。表6区分代表场景计数和另外发现的问题。

表 6: 第一版检查的口径

内容	实际结果与适用范围
原设计代表场景	32 项: 31 PASS、1 FAIL、0 BLOCKED。
其余原设计场景	71 项 NOT_RUN, 部分未覆盖变体另行注明。
运行失败	不同密码的交叉登录别名导致两个合法账号均被拒绝。
独立补充与审查	协议分类、分页查询次数、重复失败观察模板及事务可读性。
没有依据的结论	未证明生产负载表现、全面安全性或所有并发组合。

这也回答了“能编译、能运行是否等于设计正确”：第一版的主要流程能够运行，仍然对一个合法账号组合做出错误假设。查询次数和可读性问题不一定造成当前功能失败，却影响后续效率与修改风险。质量分析需要把这几种证据分清。

6.3 四个有代表性的问题

6.3.1 登录别名：字段唯一不等于入口唯一

我在登录检查中使用一个合法组合：用户甲的邮箱等于用户乙的用户名，两人的密码不同。用户名列和邮箱列各自都不重复，但共享“用户名或邮箱”入口以后，可以同时查到两个不同账号。第一版在校验密码之前就假设只能找到一个账号，因此两次正确密码登录都被拒绝。

根因不在 BCrypt 算法，也不在数据库放进了非法数据，而在字段唯一性被扩大解释为登录入口唯一。直接拒绝已有合法账号或规定用户名优先，看起来容易修复，却改变了身份语义。后面的约束因此先确认所有候选，再只在唯一密码匹配时选择身份。

6.3.2 协议异常：把错误请求误记成服务器故障

对已存在路径使用不支持的方法，或向需要 JSON 的接口发送不支持的媒体类型，第一版都返回 500。补充 HTTP 检查使用有效认证和 CSRF，前后会话查询仍成功，排除了把会话失效误当协议问题的情况。

全局错误处理已经接管框架异常，但客户端异常白名单遗漏了这两类情况，它们落入兜底内部故障分支。这里的问题是分类边界不完整，不能通过把所有异常改成 400 解决，否则真正的服务器故障又会被隐藏。

6.3.3 分页查询：单条详情的做法不适合整页

第一版把读取单条需求的组装函数用在分页结果的每一行。这一函数内部还查询标签，因此 1 行和 20 行结果页分别产生 1 次和 20 次标签 SELECT。实际查询计数支持了代码审查提出的 N+1 问题，返回内容本身仍然正确。

根因是把含 SQL 的组装步骤当作普通纯函数复用。减少 Java 代码重复不一定减少数据库工作；集合场景需要从整页考虑数据读取。这里测量的是 SQL 次数，不能把它写成已经测得响应时延或生产吞吐提升。

6.3.4 失败观察和关键事务：复制模板增加修改风险

六个写操作 facade 分别复制了失败目标定位、权限拒绝分类、独立审计和异常重抛模板；变更应用的多个重要步骤又被压在很长的单行代码中。代码能够执行，但审阅一个步骤是否仍在原事务内，以及未来修改是否漏改另一份模板，都更困难。

这是维护性和职责组织问题。现有证据没有表明第一版已经部分提交，或写入假的成功审计，因此我没有把潜在风险写成已发生的严重事务故障。修改需要集中稳定的技术规则，并让关键步骤可读，同时保持原有事务边界和业务守卫。

6.4 这些问题为什么会出现

四个问题来自不同假设：认证把数据列约束误当入口约束；错误处理只覆盖常见分支；分页把局部查询带入集合；分模块生成时横向模板被复制。它们都说明局部代码“看起来合理”与整体设计成立之间有距离。

多智能体也不能自然消除这种距离。各实现角色按模块完成任务时，更容易看到单个接口或单行数据，而测试和审查需要把模块放回共同业务过程。这里有效的是分工后的不同判断视角，以及能核对的实际观察，不是简单多生成几份代码。

7 受约束的第二次设计

7.1 从四个问题得到四条约束

如果只向 AI 说“优化认证、性能和代码质量”，它可以改输入规则、重写接口，甚至把原失败场景从需求里排除。结果可能更整齐，却不一定仍是同一个系统。因此四条约束既说明要改什么，也说明哪些含义必须保持，见表7。

表 7: 由实际根因形成的四条修改约束

编号	必须改变	必须保持
1	对每个不同用户名/邮箱候选校验，只在恰好一个密码匹配时选择身份。	原账号取值域、列唯一性、禁用检查及 401 身份拒绝；不泄露候选与哈希。
2	显式分类不支持方法和媒体类型，映射为已有 400 输入错误。	真正内部错误仍为 500，原 401、403、409 和脱敏响应不变。
3	非空需求页一次批量读取全部标签，按原顺序组装。	分页、总数、标签顺序、DTO 内容和单条详情语义不变。
4	六个 facade 共用窄失败观察契约，变更事务步骤易读。	独立失败审计、原异常、已提交清理特例和应用单事务不变。

这四条约束没有增加实体，也没有重新定义需求状态。它们把发现的根因转成能够检查的边界，避免一次优化顺手改变业务。约束本身还不是改进证据，代码和实际结果仍要再次检查。

7.2 第二版做了什么

认证读取至多两个不同身份候选，固定进行两个 BCrypt 比较，不足的候选用占位哈希补齐。恰好一个匹配后，再检查该身份启用状态；零个或多个匹配仍拒绝。这样修复了不同密码的交叉别名情形，而没有用角色或记录顺序任意选择用户。

错误处理加入两类已复现协议异常的明确映射，采用系统已有的 INVALID_INPUT/400。这与通用 HTTP 实现常见的 405、415 不同，本文说明的是本系统既定错误合约下的有界修改，不能把它描述为增加了新业务错误接口。

分页先收集本页需求标识，一次读取标签投影并分组，再按原需求顺序组装，单条详情行为保持原样。失败观察则集中到审计模块的窄契约，由调用方提供可信操作者和目标上下文；它不包围业务事务，也不接管各模块的角色、状态和自评守卫。应用变更拆成可读步骤，仍然一次提交。

7.3 V1 / V2 对比

两版使用相同的 32 个原设计场景、相同预期和运行脚本。应用 V2 为 32 项通过、零失败，与应用 V1 的 31 项通过、1 项失败比较，改进发生在原登录失败场景，而不是扩大了通过计数。表8把运行结果、补充观察和开发检查分开。

表 8: 相同口径下的两版比较

项目	应用 V1	应用 V2
原代表场景通过/失败	31 / 1	32 / 0
原设计未运行	71	71
不同密码交叉别名登录	两个正确身份被拒绝	各自密码识别各自身份
两类补充协议错误	500 内部错误	400 既有输入错误
1 行/20 行页标签 SELECT	1 / 20	1 / 1
失败观察模板	六处重复	统一窄契约
开发阶段后端检查	45 项	72 项 (含 27 项新增定向检查)
开发阶段前端检查	152 项	152 项

四处事务故障和已执行的重复、并发及权限场景继续保持预期结果。补充查询和认证边界检查为各自的问题提供证据，不能把它们加进原 32 项以后重新计算成功率。当前展示层调整后也复跑了原两条浏览器旅程；它支持展示调整没有破坏这两条流程，不改变历史 V1/V2 结果。

仍有一个无法凭现有输入解决的身份情况：两个不同账号共享同一别名，输入密码又同时匹配两者。系统没有足够信息知道用户指向谁，因此仍返回 401，不能按启用状态、角色或用户名优先级任选其一。解决它需要新的身份区分规则，而本次约束没有授权改变账号业务语义。

8 我的结论

回到题目，我对需求管理的理解从保存需求文本，转向了保存可以被确认、变更和追踪的要求。需求属性需要回答来源、理由和满足条件；ER 模型需要说明不同事实怎样归属；状态机需要说明什么时候允许改变。这三件事一起约束实现，不能由任何一张漂亮的图单独替代。

实体抽取也不等于把页面名变成表名。正式版本来自旧内容不能被覆盖，评审快照来自需要知道当时提交了什么，变更申请来自批准内容需要受控修改。先说明这些事实，十三个实体之间的关系才有理由；如果先决定表数量再补解释，模型容易变成实现结构的事后包装。

架构选择同样需要看代价。单体降低了跨模块事务难度，但不能因此宣称比微服务普遍更好；完整快照便于独立阅读，但增加了存储和复制约束；角色并集便于组合职责，但还要单独限制自评。课程中的规范化、模块职责、状态转换和事务不是互相独立的术语，它们都在帮助解释一次业务动作怎样保持含义。

AI 辅助实现最容易省略的，是看似局部合理的代码背后的整体假设。字段唯一性、异常分类、集合查询和横向模板这四个问题，均不能只靠一次编译发现。多角色

分工的价值在于让需求预期、代码实现和检查结论相互核对；若各角色只是重复同一判断，增加人数并不会自动得到独立验证。

把根因转成约束比再次笼统生成更可靠，因为它既提供修改方向，又保留已经成立的业务边界。相同场景比较使我能够解释第二版改善了哪一处，也使没有检查的范围继续保持未知。本次仍有 71 项原设计场景未运行，部分变体未覆盖，未开展生产负载、全面安全和长期可用性验证，同别名同有效密码的双身份也仍然拒绝。

本文给出独立理解所需的问题、模型、流程、设计选择、实际发现和比较结论。完整功能需求、业务规则、字段字典、外键表、接口定义和补充测试说明放在公开仓库中。它们用于复核细节，不能替代上述论证。对我而言，这次作业更重要的收获是：设计不是编码前的一次性说明，而是实现、检查和修改时仍要遵守的共同解释。

参考文献

- [1] Jeffrey L. Whitten, Lonnie D. Bentley. 系统分析与设计方法，第 7 版. 需求分析、数据建模与数据库设计相关章节。
- [2] IBM. *Creating attributes for requirement artifacts, DOORS Next 7.1.0*. <https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/doors-next/7.1.0?topic=projects-creating-attributes-requirement-artifacts>.
- [3] IBM. *Links between artifacts, DOORS Next 7.1.0*. <https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/doors-next/7.1.0?topic=artifacts-links-between>.